잇기: JOIN · 집계 · 데이터 바꾸기

메타데이터론 3강 ② · 데이터베이스와 SQL

지난주 ① '구조: 관계형 DB 읽기와 한 테이블 질의'에 이어 · 마지막에 LRM · RDF와 다시 잇기

```
FROM 지난주 JOIN 오늘
```

```
WHERE 차시 = '3강 ②'
```

```
GROUP BY 주제
```

2026년 2학기 · 전남대학교 문헌정보학과 부교수 박주현 · park51566@jnu.ac.kr

목차

1 **JOIN: 표 잇기**
INNER · LEFT · 다대다 → 실습 3 (25분)

2 **집계**
COUNT · AVG · GROUP BY · HAVING

3 **하위 질의와 날짜**
질의 안의 질의 · julianday → 쉬는 시간 · 실습 4 (30분)

4 **데이터 바꾸기**
INSERT · UPDATE · DELETE · 제약 · 트랜잭션 → 실습 5 (15분)

5 **메타데이터 관점**
book · item · work와 LRM · 표와 그래프 → 생각 2 (10분)

지난주 · 3강 ① 구조

1 왜 데이터베이스인가
2 데이터 모델
3 관계형 모델의 말
4 키
5 ERD와 관계
6 정규화
7 SQL 개요
8 SELECT

오늘(②): 1부 → 실습 3 → 2·3부 → 쉬는 시간 → 실습 4 → 4부 → 실습 5 → 5부 → 생각 2

복습: 나눠 담고 번호로 가리킨다

표 · 키

- 행 하나 = 레코드 하나
- PK: 행을 하나로 가리킨다
- FK: 다른 표의 PK를 가리킨다

ERD · 정규화

- 1:N은 FK로 (book → item)
- N:M은 잇는 표로 (book_author)
- 같은 사실은 한 곳에만

SELECT

- `SELECT 열 FROM 표`
- `WHERE · ORDER BY · LIKE`
- `IS NULL · COUNT · DISTINCT`

오늘: 나눠 둔 표를 다시 잇는다 → 1부 JOIN

지난주 12·17·20번(표·키) · 22·24·29번(관계·정규화) · 36~41번(SELECT) 요약

나눠 두었으니 다시 이어야 한다

item: 번호만 보인다

item_id	book_id	location	status
EM000122	2	2층 인문자료실	대출중
EM000129	2	2층 인문자료실	대출가능
EM000136	2	보존서고	대출중



book

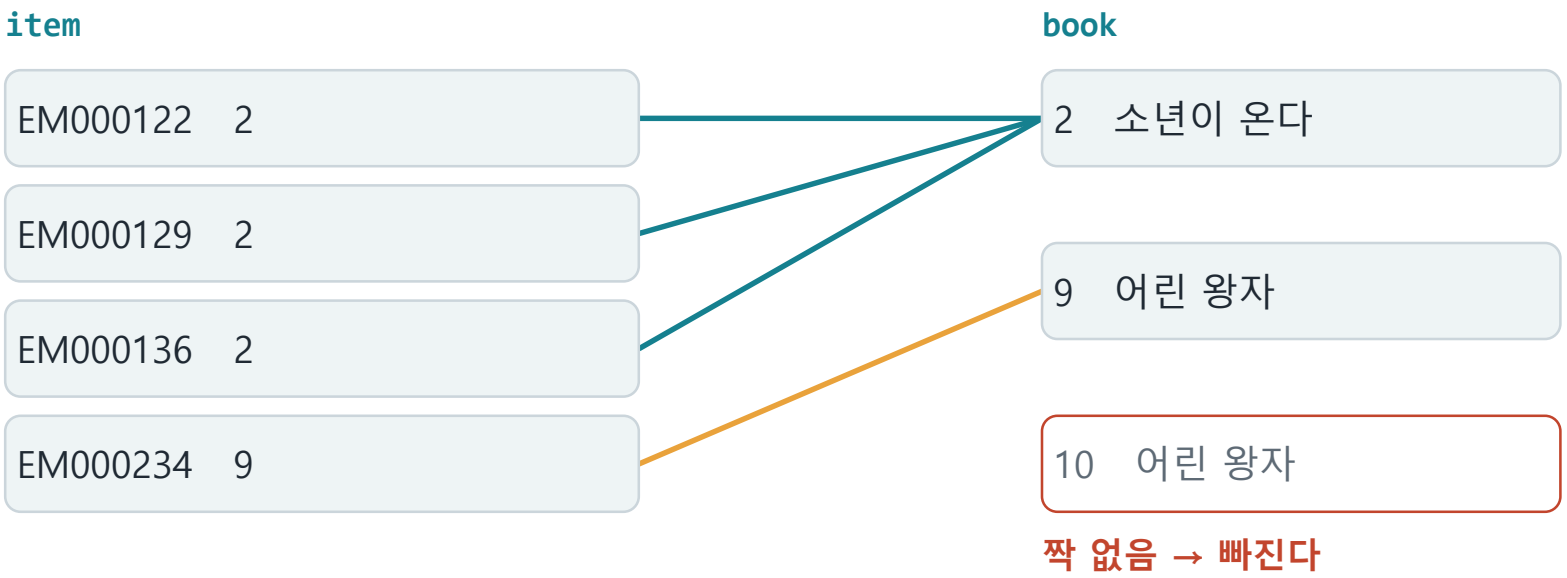
book_id	title
2	소년이 온다

2번이 무슨 책?

정규화로 나눴으면, 볼 때는 JOIN으로 잇는다

값: library.sql item(book_id = 2) · book 2번

JOIN의 원리: 같은 값끼리 짝짓기



짝지은 결과 (4행)

item_id	book_id	title
EM000122	2	소년이 온다
EM000129	2	소년이 온다
EM000136	2	소년이 온다
EM000234	9	어린 왕자

INNER JOIN: 양쪽 모두 짝이 있는 행만 · 짝 조건은 ON 뒤에

값: library.sql item 4행과 book 2·9·10번 (설명용으로 골라 놓음)

JOIN 써 보기: FROM · JOIN · ON

예시 질의

```
SELECT i.item_id, i.location, b.title
FROM item i
JOIN book b ON b.book_id = i.book_id
WHERE b.book_id = 2;
```

실행 결과

item_id	location	title
EM000122	2층 인문자료실	소년이 온다
EM000129	2층 인문자료실	소년이 온다
EM000136	보존서고	소년이 온다

3행

`item i · book b` — 별칭: 표 이름을 짧게

`ON b.book_id = i.book_id` — 짝 조건 (FK = PK)

`b.title` — 어느 표의 열인지 밝힌다

별칭을 안 붙이면

`ambiguous column name: book_id`
= 두 표에 다 있는 열

결과·메시지: library.sql을 SQLite로 실행 (메시지는 SELECT book_id FROM book JOIN item ON book.book_id = item.book_id)

LEFT JOIN: 짝이 없어도 왼쪽은 남긴다

INNER JOIN (그냥 JOIN)

```
SELECT m.name, r.reserved_on
FROM member m
JOIN reservation r
  ON r.member_id = m.member_id
WHERE m.member_type = '교원';
```

실행 결과

name	reserved_on
권태호	2026-10-25
황미경	2026-11-03

2행 — 안준영이 없다

LEFT JOIN

```
SELECT m.name, r.reserved_on
FROM member m
LEFT JOIN reservation r
  ON r.member_id = m.member_id
WHERE m.member_type = '교원';
```

실행 결과

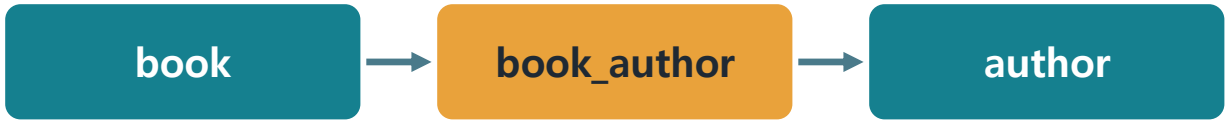
name	reserved_on
권태호	2026-10-25
황미경	2026-11-03
안준영	*NULL*

3행 — 짝 없는 칸은 NULL

LEFT JOIN 뒤 오른쪽 칸 IS NULL = '한 번도 ~하지 않은' 것 찾기

결과: library.sql을 SQLite로 실행 · 이용자는 가상

다대다 잇기: 잇는 표를 거쳐 세 표를



JOIN ①: 책 번호　　JOIN ②: 사람 번호

예시 질의

```
SELECT b.title, a.name, ba.role
FROM book b
JOIN book_author ba ON ba.book_id = b.book_id
JOIN author a       ON a.author_id = ba.author_id
WHERE b.book_id = 9;
```

실행 결과

title	name	role
어린 왕자	앙투안 드 생텍쥐페리	지은이
어린 왕자	김화영	옮긴이

잇는 표는 거쳐 간다

N:M 하나 = JOIN 두 번
책 → 사람도, 사람 → 책도 같은 길

실습 3의 2-3·2-4는 반대 방향: 사람에서 출발해 책으로

결과: library.sql을 SQLite로 실행

실습 80분 · 브라우저 + 인터넷

실습 안내: 오늘의 실습 · 순서 · 시간

순서	내용	분	언제
실습 3	JOIN 5문제 (2-1 ~ 2-5)	25	지금 (1부 뒤)
실습 4	집계 · 하위 질의 · 날짜 7문제 (2-6 ~ 2-12)	30	쉬는 시간 뒤
실습 5	INSERT · UPDATE · DELETE 3문제 (2-13 ~ 2-15) + 도전	15	4부 뒤
생각 2	저작은 어디에 있나 (채점 없음)	10	5부 가운데

실습 5 주의

INSERT · UPDATE · DELETE는 실습 DB를 바꾼다

꼬이면 'DB 초기화'

채점은 처음 상태 DB에서 다시 돌려 비교

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

실습 3 · 25분

실습 3: JOIN 5문제 (2-1 ~ 2-5)

번호	제목	잇는 길
2-1	도서와 출판사 잇기	book — publisher
2-2	분류명 붙이기	book — kdc
2-3	세 테이블 잇기	book — book_author — author
2-4	옮긴이 찾기	2-3과 같은 길 · 역할만 다르게
2-5	한 번도 안 빌린 책	book — item — loan

막히면

- 6번: FROM · JOIN · ON 꼴
- 8번: 잇는 표 거치기
- 7번: LEFT JOIN … IS NULL
- ERD를 옆 창에 (structure.html)

2-5: 대출은 도서가 아니라 실물(item)에 붙는다

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

집계 함수: 여러 행을 숫자 하나로

예시 질의

```
SELECT COUNT(*), MIN(birth_year), MAX(birth_year),
       ROUND(AVG(birth_year), 1)
FROM author;
```

실행 결과

COUNT(*)	MIN(birth_year)	MAX(birth_year)	ROUND(AVG(…), 1)
47	1877	1990	1947.0

함수	하는 일
COUNT(*)	행 개수
SUM(열)	합계
AVG(열)	평균
MIN(열) · MAX(열)	가장 작은 · 큰 값
ROUND(값, 1)	소수 첫째 자리로 반올림

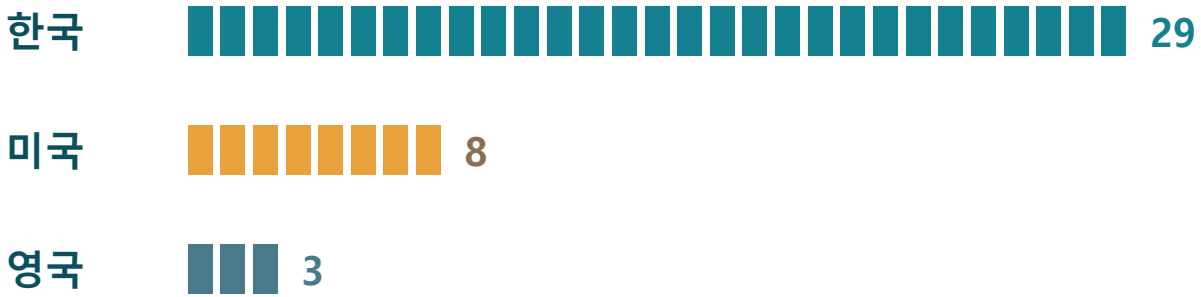
COUNT(*) 말고는 NULL을 빼고 계산한다 — 평균은 29명의 평균

결과: library.sql을 SQLite로 실행 · birth_year가 적힌 저자 29명(지난주 41번)

GROUP BY: 묶어서 세기

예시 질의

```
SELECT nationality, COUNT(*)
FROM author
GROUP BY nationality
ORDER BY COUNT(*) DESC, nationality;
```



실행 결과

nationality	COUNT(*)
한국	29
미국	8
영국	3
일본	2
독일	1
브라질	1
이스라엘	1
중국	1
프랑스	1

9행 (국적 9가지)

SELECT에는 묶은 열과 집계 함수만

결과: library.sql을 SQLite로 실행 · 막대 하나 = 저자 한 명(위 3개 국적만 그림)

HAVING: 묶은 뒤에 거르기

예시 질의

```
SELECT nationality, COUNT(*)
FROM author
GROUP BY nationality
HAVING COUNT(*) >= 3
ORDER BY COUNT(*) DESC;
```

WHERE 묶기 전 · 행 하나하나를 거른다

HAVING 묶은 뒤 · 묶음 하나하나를 거른다

실행 결과

nationality	COUNT(*)
한국	29
미국	8
영국	3

WHERE COUNT(*) >= 3 은?

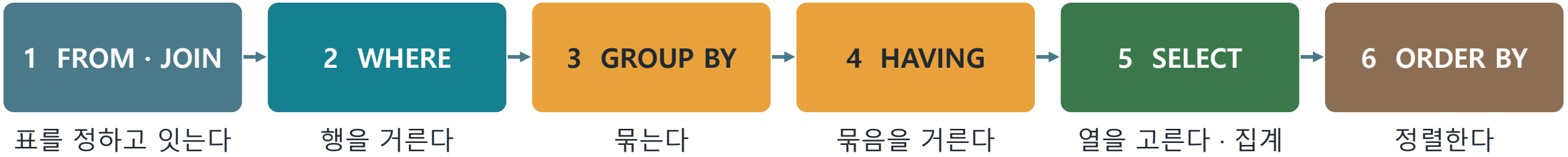
`misuse of aggregate: COUNT()`

개수는 묶어 봐야 안다 → HAVING

결과: library.sql을 SQLite로 실행

SELECT 문 전체: 쓰는 순서와 도는 순서

도는 순서



쓰는 순서 (틀리면 문법 오류)

```
SELECT   열, 집계함수(…)
FROM     표 a
JOIN     표 b ON 짝 조건
WHERE    행 조건
GROUP BY 묶을 열
HAVING   묶음 조건
ORDER BY 정렬 기준;
```

그래서

- WHERE에서는 COUNT를 못 쓴다 (아직 안 묶음)
- SELECT에서 붙인 별칭은 ORDER BY에서 쓸 수 있다 (SELECT 다음)

하위 질의: 질의 안의 질의

예시 질의

```
SELECT name FROM member
WHERE member_id IN (
  SELECT member_id FROM reservation
  WHERE status = '대기'
);
```

꼴	쓰는 곳
열 IN (SELECT …)	목록 안에 있나
열 > (SELECT AVG(…) …)	값 하나와 비교
EXISTS (SELECT …)	하나라도 있나 (NOT EXISTS: 하나도 없나)

실행 결과

name
김하늘
윤채원
오수아
서유진
권태호
황미경

6행 (이용자는 가상)

① 괄호 안이 먼저: 대기 예약의 이용자 번호들
② 바깥: 그 번호의 이름

결과: library.sql을 SQLite로 실행

날짜 계산: 글자로 적고 함수로 셈한다

예시 질의 ①: 며칠째 기다리나

```sql
SELECT reservation_id, reserved_on,
       julianday('2026-11-05') - julianday(reserved_on) AS days
FROM reservation
WHERE status = '대기';
```

예시 질의 ②: 14일 뒤

```sql
SELECT date('2026-11-05', '+14 days');
```

→ **2026-11-19**

실행 결과

reservation_id	reserved_on	days
4	2026-10-20	16.0
5	2026-10-22	14.0
6	2026-10-25	11.0
8	2026-10-28	8.0
10	2026-11-01	4.0
11	2026-11-03	2.0

6행

'YYYY-MM-DD' 글자 = 비교·정렬 그대로 · 날짜 함수는 DBMS마다 다르다

결과: library.sql을 SQLite로 실행 · 기준일 2026-11-05 · AS days = 결과 열에 붙인 별칭

쉬는 시간 10분

다음: 실습 4 (2-6 ~ 2-12, 30분) → 4부 데이터 바꾸기 → 실습 5

쉬는 동안: 실습 3에서 남은 문제 이어 풀기
parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/lab.html

실습 4 · 30분

실습 4: 집계 · 하위 질의 · 날짜 7문제 (2-6 ~ 2-12)

번호	제목	쓰는 것	순서 채점
2-6	유형별 대출 건수	JOIN · GROUP BY	○
2-7	많이 빌린 책 (HAVING)	JOIN 둘 · HAVING	○
2-8	주류별 도서 수	JOIN · GROUP BY	○
2-9	같은 서명, 다른 판	GROUP BY · HAVING	
2-10	연체 중인 대출	JOIN 셋 · 날짜 비교	○
2-11	평균보다 두꺼운 책	하위 질의 · AVG	
2-12	평균 대출 기간	julianday · ROUND · AVG	

막히면

- 12·13번: GROUP BY · HAVING
- 14번: 쓰는 순서
- 15·16번: 하위 질의 · 날짜
- 기준일 2026-11-05

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

INSERT: 행 넣기

예시: 넣고 확인하기

```
INSERT INTO kdc VALUES ('020', '문헌정보학', '000 총류');

SELECT * FROM kdc WHERE code = '020';
```

실행 결과

code	name	main_class
020	문헌정보학	000 총류

1행 — kdc는 17행 → 18행

열 이름을 적는 꼴 (적지 않은 열 = 기본값 · NULL)

```
INSERT INTO 표 (열1, 열2, …)
VALUES (값1, 값2, …);
```

값 적는 법

- 글자 · 날짜: '작은따옴표'
- 숫자: 그대로
- 비울 때: NULL (따옴표 없이)
- VALUES 값 순서 = 열 순서

결과: library.sql을 SQLite로 실행 · KDC 020 = 문헌정보학

UPDATE · DELETE: WHERE가 생명줄

예시 ①: 한 권 옮기기 (1행 바뀜)

```
UPDATE item SET location = '보존서고'
WHERE item_id = 'EM000178';
```

예시 ②: 19번에서 넣은 행 지우기 (1행)

```
DELETE FROM kdc WHERE code = '020';
```

1. **같은 WHERE로 SELECT** 몇 행이 걸리나?
2. **UPDATE · DELETE** 같은 WHERE 그대로
3. **다시 SELECT** 바뀐 것 확인

WHERE를 빼면

```
DELETE FROM reservation;
```

→ 예약 11행이 모두 지워진다
실제 DB에는 'DB 초기화'가 없다

바꾸기 전에 같은 WHERE로 SELECT

결과: library.sql을 SQLite로 실행(바뀐 행 1개씩) · reservation은 11행

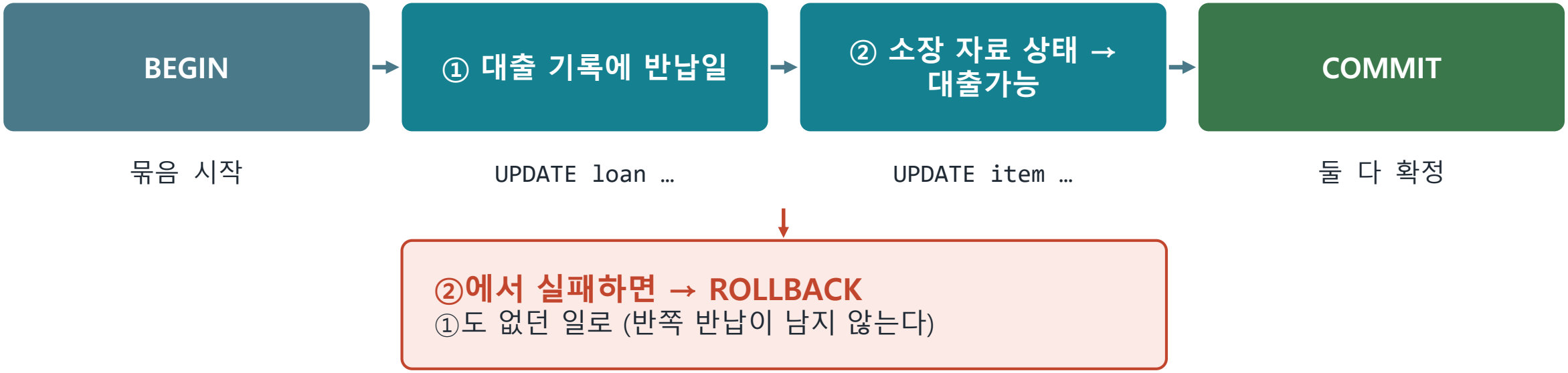
무결성 제약: DB가 막아 주는 실수

이렇게 하면	SQLite 메시지	지킨 규칙
kdc에 '020'을 두 번 넣기	`UNIQUE constraint failed: kdc.code`	기본 키는 겹칠 수 없다
publisher에 '창비'를 또 넣기	`UNIQUE constraint failed: publisher.name`	UNIQUE (대체 키)
이름 없이 member 넣기	`NOT NULL constraint failed: member.name`	필수 칸
없는 도서 99번의 item 넣기	`FOREIGN KEY constraint failed`	참조 무결성: 없는 번호
item이 가리키는 book 1번 지우기	`FOREIGN KEY constraint failed`	참조 무결성: 가리키는 행

무결성 제약 = 필수 요소 검사 · 전거 연결 검사를 DB가 대신

메시지: library.sql을 Python sqlite3(SQLite 3.50.4)로 실행해 받음 · 실습실(sql.js)도 SQLite 메시지를 보여 준다

트랜잭션: 모두 되거나, 하나도 안 되거나



트랜잭션 = 여러 문장을 한 덩어리로 · 전부 되거나 하나도 안 되거나

'…'는 생략 · 실습 5의 2-14(반납 처리)가 이 두 단계

실습 5 · 15분

실습 5: 데이터 바꾸기 3문제 + 도전 (2-13 ~ 2-15)

번호	제목	쓰는 것
2-13	새 이용자 넣기	`INSERT`
2-14	반납 처리 (UPDATE 두 번)	`UPDATE × 2 · 하위 질의`
2-15	취소된 예약 지우기	`DELETE … WHERE`
도전	서평 테이블 설계 (채점 없음)	`CREATE TABLE · CHECK`

바꾸기 전에 같은 WHERE로 SELECT · 꼬이면 'DB 초기화'

도전: review 테이블

- 누가 (이용자)
- 무엇에 (도서)
- 언제 · 몇 점(1~5) · 어떤 글
- 외래 키로 잇고, 한 행 넣고, JOIN으로 보기

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

작은 도서관 DB를 LRM으로 읽기

LRM 개체	작은 도서관 DB	예
저작 (Work)	**없음**	→ 생각 2
표현형 (Expression)	없음 (language · 옮긴이가 실마리)	김화영 번역 · 황현산 번역
구현형 (Manifestation)	**book — 판 하나**	문학동네 2007 · 열린책들 2015
개별자료 (Item)	**item — 실물 한 권**	EM000234 · EM000241
에이전트 (Agent)	**author + book_author.role**	생텍쥐페리(지은이) · 김화영(옮긴이)

판(구현형)과 실물(개별자료)은 있지만, 작품(저작)은 DB 어디에도 없다

LRM 용어: 1장 덱 표기 · 값: library.sql book 9·10번, item, author 7·8·9번

『어린 왕자』 세 종: DB는 같은 작품인 줄 모른다

book · publisher · book_author · author · item을 이어 본 결과

book_id	title	출판사	pub_year	옮긴이	language	실물
9	어린 왕자	문학동네	2007	김화영	kor	2
10	어린 왕자	열린책들	2015	황현산	kor	1
31	The Little Prince	Harcourt	2000	리처드 하워드	eng	1

서명으로 묶으면
9 · 10만 묶이고 31은 빠진다

서명이 같아도
같은 작품이라는 보장은 없다

번역이 다르면
표현형이 다르다 (김화영 · 황현산 · 하워드)

작품을 가리키는 식별자가 없으면, DB는 같은 작품을 모을 수 없다

결과: library.sql을 SQLite로 실행(book 9·10·31번, 실물 = item 행 수) · 서명 묶음은 실습 4의 2-9

생각 2 · 10분 · 채점 없음

생각 2: 저작은 어디에 있나

2-9의 결과를 LRM으로 읽어 보세요. book 한 행은 저작 · 표현형 · 구현형 · 개별자료 가운데 무엇에 가깝고, item 한 행은 무엇인가요? '어린 왕자' 세 종을 한 저작으로 묶으려면 어떤 테이블과 열이 더 필요할까요?

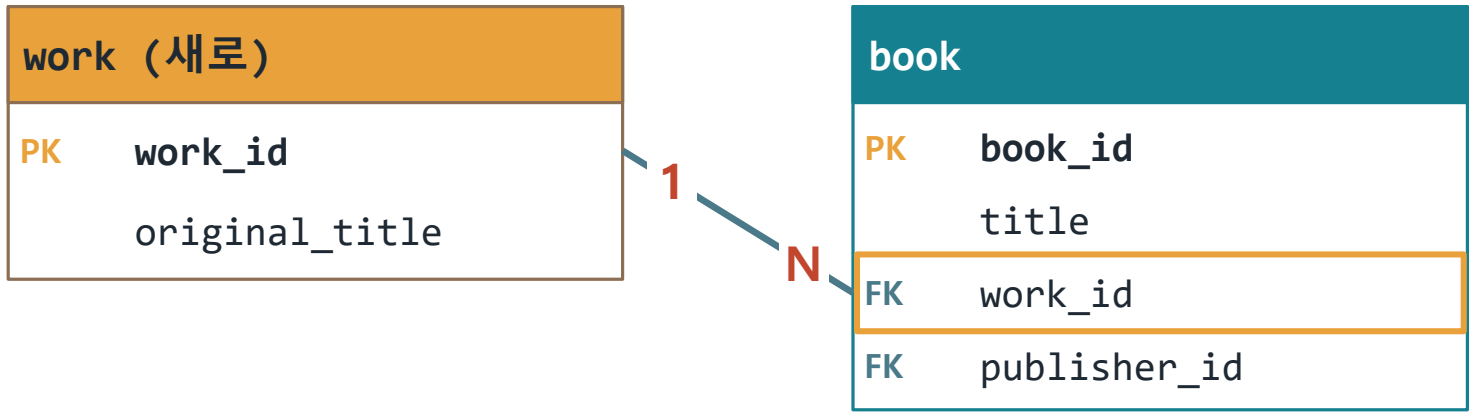
힌트 1
서명이 같다고 같은 저작이 아니고, 서명이 달라도 같은 저작일 수 있다

힌트 2
work(저작) 테이블을 두고 book에 work_id를 붙이는 설계를 그려 보기

종이에 ERD로 그리기: 상자 · 열 · 1과 N

문제: 실습실 '생각 2' (exercises.json, 힌트 포함)

설계안 하나: work 테이블을 더하면



work (예시 행)

work_id	original_title
1	Le Petit Prince

book에 work_id를 채우면

book_id	title	work_id
9	어린 왕자	1
10	어린 왕자	1
31	The Little Prince	1

더 나가면

work 1 — N expression
언어 · 번역자

expression 1 — N book
판

book 1 — N item
실물

설계 예시 — 실습 DB(library.sql)에는 work · expression 테이블이 없다 · 원제는 2강 book.json의 originalTitle

같은 사실, 표와 그래프

관계형: 표의 한 행

book_id	author_id	role
9	8	옮긴이

번호 9 · 8은 이 DB 안에서만 통한다

RDF: 트리플 하나 (2강)

```
ex:book9  ex:translator  ex:author8 .
```

URI라서 다른 기관 데이터와 바로 잇는다

	관계형 DB	RDF 그래프
틀	스키마를 먼저 정한다	트리플을 계속 더한다
가리키기	키 (DB 안)	URI (웹 전체)
잇기 · 묻기	JOIN · SQL	링크 따라가기 · SPARQL

ex:는 수업용 예시 어휘 · 값: library.sql book_author (9, 8, '옮긴이')

같은 질문, SQL과 SPARQL

SQL (실습 DB에서 실행)

```sql
SELECT b.title
FROM book b
JOIN book_author ba ON ba.book_id = b.book_id
JOIN author a ON a.author_id = ba.author_id
WHERE a.name = '김화영'
  AND ba.role = '옮긴이';
```

SPARQL (수업용 예시 어휘 ex:)

```sparql
PREFIX ex: <http://example.org/>
SELECT ?title
WHERE {
  ?book ex:translator ?p .
  ?p    ex:name "김화영" .
  ?book ex:title ?title .
}
```

실행 결과

title
어린 왕자

1행

닮은 점 · 다른 점

SELECT · WHERE는 같다 · JOIN 대신 같은 변수(?book · ?p)를 되풀이

SQL을 읽으면 SPARQL도 읽힌다 — 표를 잇느냐, 그래프를 따라가느냐

SQL 결과: library.sql을 SQLite로 실행 · SPARQL은 설명용(실행 결과 없음)

두 주 정리: 구조 → 질의 → 잇기 → 바꾸기 → 다시 보기



메타데이터 설계 = 무엇을 한 곳에 적고, 무엇으로 가리킬지 정하는 일

실습실은 수업 뒤에도 열려 있다: parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

감사합니다

질문과 의견을 자유롭게 나눠 주세요

전남대학교 문헌정보학과 부교수 박주현 · park51566@jnu.ac.kr